Incidentes y soluciones de Docker, PostgreSQL y cuentas

- Incidentes y soluciones de Docker, PostgreSQL y cuentas
  - Resumen del estado
  - Cambios realizados en las cuentas y la base de datos
  - Problemas encontrados y cómo se resolvieron
  - Comandos útiles

Este documento registra los problemas que aparecieron al preparar NutriVida en Windows, cómo se identificó la causa y qué quedó funcionando. Los errores descritos no significan que el código de la aplicación estuviera necesariamente fallando: algunos ocurrieron en WSL, Docker Desktop o en la selección de la dirección local.

Resumen del estado

- Docker Desktop inicia y ejecuta los cuatro servicios: PostgreSQL, usuarios, administración y frontend.
- PostgreSQL está saludable y conserva sus datos en el volumen de Docker `usuarios-db`.
- La interfaz Docker se abre en http://127.0.0.1:5173.
- Se crearon cinco cuentas de prueba: dos profesionales y tres pacientes, además de la cuenta administradora.
- Cada cuenta de prueba tiene una contraseña distinta. Las credenciales están en `.cuentas-prueba.txt`, archivo local que no se debe compartir.
- Se probó el inicio de sesión de las cinco cuentas a través del frontend Docker.

Cambios realizados en las cuentas y la base de datos

Situación inicial

Al iniciar Docker por primera vez, `.env` tenía `CARGAR_DATOS_INICIALES=false`. Por ese motivo no se habían creado las cuentas de prueba de pacientes ni profesionales: esos usuarios aún no tenían correo ni contraseña para iniciar sesión. El servicio sí crea por separado la cuenta administrativa usando `ADMIN_EMAIL` y `ADMIN_PASSWORD`.

No se inventaron contraseñas para cuentas que ya existieran ni se cambió la cuenta de administración. Se habilitó la carga de demostración y se crearon las cinco cuentas faltantes en la base PostgreSQL que ya estaba en uso.

Cuentas de demostración creadas

Rol	Correo para iniciar sesión	Datos asociados
Administración	`admin@nutrivida.cl`	Cuenta preexistente, conservada sin cambios.
Profesional	`valentina.rojas@nutrivida.cl`	Nutricionista; tiene asignados a Esteban Zavala y Juan López.
Profesional	`matias.fuentes@nutrivida.cl`	Nutricionista; tiene asignado a Camilo Calderón.
Paciente	`esteban.zavala@nutrivida.cl`	Una medición inicial de demostración.
Paciente	`juan.lopez@nutrivida.cl`	Tres mediciones de demostración.
Paciente	`camilo.calderon@nutrivida.cl`	Tres mediciones de demostración.

Los cinco usuarios nuevos están activos. La base pasó a contener una cuenta `ADMIN`, dos `PROFESIONAL` y tres `PACIENTE`, con siete mediciones de ejemplo. No se eliminó ni se reemplazó el volumen PostgreSQL.

Cómo se manejaron las contraseñas

Se generó una contraseña aleatoria diferente para cada una de las cinco cuentas nuevas. Los valores se guardaron únicamente en el archivo local ignorado por Git `.cuentas-prueba.txt`, en la raíz de `fase3`. Ese archivo sirve para acceder a las cuentas de prueba en este equipo; no se incluye en la documentación ni se debe enviar junto a un ZIP público.

La aplicación guarda en PostgreSQL el hash BCrypt (`passwordHash`), no la contraseña legible. La contraseña legible existe en el archivo local solo para entregar las credenciales de prueba a la persona autorizada que las necesite. Compártelas únicamente por un canal privado y cámbialas después de la demostración. La contraseña de administración se conserva en `.env`; no se copia al archivo de cuentas de prueba ni a esta documentación.

Reproducir la carga en una instalación nueva

La base PostgreSQL se conserva en un volumen local de Docker, no dentro del código fuente. Por eso, quien reciba una copia del proyecto no recibe automáticamente las filas de la base ni las contraseñas locales de este equipo.

Para crear las cuentas de demostración en una instalación nueva:

1. Copia `.env.example` como `.env` y define secretos propios para PostgreSQL, JWT y administración.
2. Cambia `CARGAR_DATOS_INICIALES=false` a `CARGAR_DATOS_INICIALES=true`.
3. Define las cinco variables `CLAVE_INICIAL_*` en `.env` con contraseñas distintas, seguras y que se comunicarán privadamente a los evaluadores. No reutilices las claves de este equipo.
4. Ejecuta `docker compose up --build -d` desde la carpeta `fase3`.
5. Inicia sesión en http://127.0.0.1:5173 y luego vuelve a dejar `CARGAR_DATOS_INICIALES=false` en `.env`. La carga no cambia contraseñas de cuentas cuyo correo ya exista.

La copia de `.env.example` viene con la carga desactivada intencionalmente para que los datos de demostración no se agreguen de manera inesperada. Para una entrega con demostración, hay que seguir los pasos anteriores y proporcionar las credenciales por separado y de forma segura.

Problemas encontrados y cómo se resolvieron

1. WSL indicaba `REGDB_E_CLASSNOTREG`

Qué se observó: `wsl --status` mostró que la instalación de WSL estaba dañada y que una clase no estaba registrada. Mientras Docker Desktop no podía iniciar, Docker mostraba el cliente pero no encontraba su servidor o su canal de comunicación.

Causa: Docker Desktop necesitaba que WSL 2 estuviera operativo para iniciar su motor Linux.

Cómo se manejó: se dejó que WSL reparara/actualizara su instalación y se volvió a ejecutar `wsl --status`. Después, el comando devolvió la versión predeterminada de WSL 2. La ausencia de una distribución Linux personal no impedía por sí sola que Docker Desktop usara su entorno propio.

Comprobación: `docker version` debe mostrar tanto `Client` como `Server`, y `docker compose ps` debe poder comunicarse con el motor.

2. Docker mostraba el cliente, pero el motor no estaba disponible

Qué se observó: errores como “Docker Desktop is unable to start” o “The system cannot find the file specified” para `dockerDesktopLinuxEngine`.

Causa: el comando `docker` estaba instalado, pero Docker Desktop todavía no había iniciado el motor y no existía el canal que usa el cliente para comunicarse con él.

Cómo se manejó: se abrió Docker Desktop, se esperó a que indicara que el motor estaba en ejecución y se comprobó de nuevo con `docker version`. `wsl --shutdown` puede utilizarse como parte de un reinicio de WSL, pero no sustituye la reparación de una instalación dañada ni el inicio de Docker Desktop.

Comprobación:

```
docker version
docker compose version
docker compose ps
```

No se deben confundir comandos concatenados, por ejemplo `docker versiondocker version`, con dos comandos separados.

3. Windows no permitió publicar los puertos 8081 y 8082

Qué se observó: Docker no pudo enlazar los puertos que usan internamente las API.

Causa: esos puertos no estaban disponibles para su publicación en el equipo Windows.

Cómo se manejó: Compose permite elegir los puertos del equipo mediante `USUARIOS_HOST_PORT`, `ADMIN_HOST_PORT` y `FRONTEND_HOST_PORT`, manteniendo sin cambios los puertos internos de cada contenedor. En la configuración local quedaron `8083` para usuarios, `8084` para administración y `5173` para el frontend.

Los servicios se publican en `127.0.0.1`, por lo que son accesibles desde el propio equipo y no se publican directamente a la red local.

4. No aparecían cuentas de pacientes ni profesionales

Qué se observó: se podía iniciar sesión con la cuenta administrativa, pero no existían las cuentas de prueba de pacientes y profesionales.

Causa: la carga opcional estaba configurada como `CARGAR_DATOS_INICIALES=false`. El arranque normal crea la cuenta administrativa, pero no crea automáticamente las cinco cuentas de demostración. La opción de carga estaba apagada intencionalmente para no agregar datos de prueba sin pedirlo.

Cómo se manejó: se activó la carga inicial y se generó una contraseña aleatoria diferente para cada profesional y paciente. Las contraseñas se guardaron en el archivo local `.cuentas-prueba.txt`; no se escribieron en el README ni en el código. Se conservó la cuenta administradora y el volumen existente de PostgreSQL.

Comprobación: la tabla de usuarios mostró una cuenta `ADMIN`, dos `PROFESIONAL` y tres `PACIENTE`, todas activas. Se verificó el inicio de sesión de cada una.

Importante sobre futuras cargas: la inicialización es idempotente para estas cuentas: si un correo ya existe, no vuelve a crear esa cuenta ni cambia su contraseña. Para cambiar una contraseña existente se debe usar la función de cambio/restablecimiento de contraseña, no volver a ejecutar la carga inicial. Las variables `CLAVE_INICIAL_*` permiten configurar contraseñas distintas al crear cuentas nuevas; si se dejan sin definir, Compose usa `ADMIN_PASSWORD` como alternativa.

El archivo de credenciales local contiene contraseñas en texto legible. No se debe subir, enviar ni compartir. Después de probar, cambia las contraseñas desde la aplicación y guarda las nuevas en un gestor de contraseñas. En una instalación real no se deben reutilizar estas cuentas ni sus datos de demostración.

5. El login devolvía “Credenciales incorrectas” desde `localhost`

Qué se observó: el mismo correo y contraseña funcionaban directamente contra el servicio de usuarios, pero una prueba a `http://localhost:5173` devolvía 401.

Causa: había un servidor Vite local de Windows escuchando y el navegador/comando dirigido a `localhost:5173` llegaba a ese servidor, no al Nginx del contenedor Docker. El contenedor estaba correctamente publicado en IPv4.

Cómo se manejó: se compararon las respuestas de `localhost` y `127.0.0.1`. Se confirmó que `localhost` servía el cliente de Vite y que `127.0.0.1` servía Nginx desde Docker. Se repitió el login usando la URL Docker correcta.

Resultado: las cinco cuentas de prueba iniciaron sesión a través del proxy del frontend Docker. Para usar esta instalación, abre http://127.0.0.1:5173; no uses `localhost` mientras siga activo el servidor Vite local.

6. El registro del frontend muestra avisos de React

Qué se observó: `.local/frontend-error.log` registra que React no reconoce las propiedades JSX `allowfullscreen` y `referrerpolicy` y recomienda `allowFullScreen` y `referrerPolicy`.

Impacto: son avisos de consola relacionados con los atributos del `iframe` de Google Maps en `frontend/src/pages/publico/Inicio.jsx`. No explican el fallo de login ni impidieron compilar o abrir el sitio.

Estado: queda como limpieza menor de frontend; no se cambió código durante esta revisión. La recomendación es ajustar el casing JSX y volver a ejecutar lint, pruebas y build.

7. Maven no pudo reemplazar el JAR al finalizar `verify`

Qué se observó: durante la revisión, todas las pruebas Java terminaron correctamente, pero la fase de empaquetado falló al intentar renombrar `usuarios-service-1.0.0.jar` a `usuarios-service-1.0.0.jar.original`.

Causa: Windows no permitió la operación sobre ese archivo. Esto suele ocurrir cuando otro proceso mantiene abierto el artefacto; no se confirmó qué proceso lo retenía.

Comprobación alternativa: `mvn test` ejecutó las pruebas de ambos servicios y terminó con éxito: 10 pruebas de usuarios y 4 de administración, sin fallos ni errores. El empaquetado Docker del servicio de usuarios también se realizó durante la creación de las cuentas.

Estado: las pruebas están aprobadas; la verificación Maven que incluye empaquetado no se pudo completar en ese intento. Antes de volver a ejecutar `mvn verify`, cierra de forma normal cualquier sesión local que esté ejecutando ese JAR. No se detuvieron procesos ni se borraron artefactos durante esta revisión.

Comandos útiles

Desde PowerShell, en la carpeta `fase3`:

```
docker version
wsl --status
docker compose ps
docker compose logs --tail=100 usuarios
docker compose logs --tail=100 postgres
```

Usa http://127.0.0.1:5173 para la interfaz Docker. Para detener los servicios sin borrar la base:

```
docker compose down
```

No uses `docker compose down -v` salvo que quieras eliminar deliberadamente el volumen y perder los datos de PostgreSQL.